

ООП 2

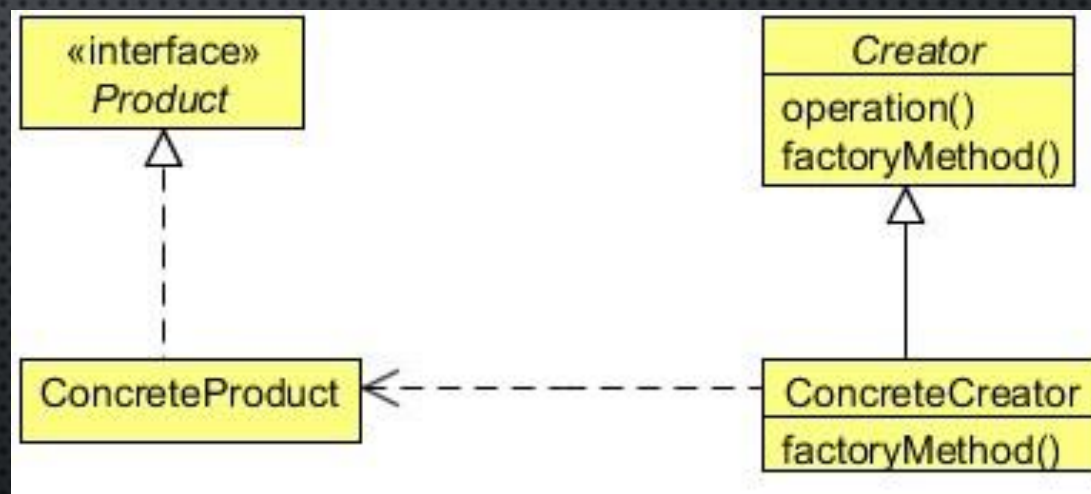
ШАБЛОНИ ЗА ДИЗАЙН. ШАБЛОНИ ЗА СЪЗДАВАНЕ 2

# МЕТОД ФАБРИКА (FACTORY METHOD)

- КАПСУЛИРА СЪЗДАВАНЕТО (ИНСТАНЦИРАНЕ) НА КОНКРЕТНИ ТИПОВЕ
  - КАПСУЛИРА ФУНКЦИОНАЛНОСТТА, НЕОБХОДИМА ЗА ИЗБОР НА ПОДХОДЯЩА ИНСТАНЦИЯ, ВЪТРЕ В ОПРЕДЕЛЕН МЕТОД
- ЕЛИМИНИРА НЕОБХОДИМОСТТА ОБЕКТ НА ПРИЛОЖЕНИЕ ДА СЕ ТОВАРИ С ДОПЪЛНИТЕЛНИ ЛОГИКИ ЗА КРИТЕРИИ ЗА ИЗБОР НА КЛАС
- СЪЗДАТЕЛИ (ФАБРИКИ), ПРОДУКТИ

# МЕТОД ФАБРИКА (FACTORY METHOD)

- ИНТЕРФЕЙС НА ПРОДУКТИТЕ
- КОНКРЕТНИ ПРОДУКТИ
- СЪЗДАТЕЛ НА ПРОДУКТИ (АБСТРАКЦИЯ)
- КОНКРЕТНИ СЪЗДАТЕЛИ



# МЕТОД ФАБРИКА (FACTORY METHOD)

- ПРОБЛЕМ
  - XML ПАРСЪР

# МЕТОД ФАБРИКА (FACTORY METHOD) ИНТЕРФЕЙС

- ```
PUBLIC INTERFACE XMLPARSER {  
    PUBLIC STRING PARSE();  
}
```

# МЕТОД ФАБРИКА (FACTORY METHOD) КОНКРЕТНИ ПРОДУКТИ

- ```
PUBLIC CLASS ERRORXMLPARSER IMPLEMENTS XMLPARSER{  
    @OVERRIDE  
    PUBLIC STRING PARSE()  
    .....  
}
```

# МЕТОД ФАБРИКА (FACTORY METHOD) КОНКРЕТНИ ПРОДУКТИ

- ```
PUBLIC CLASS FEEDBACKXML IMPLEMENTS XMLPARSER{  
    @OVERRIDE  
    PUBLIC STRING PARSE() {  
        .....  
    }
```

# МЕТОД ФАБРИКА (FACTORY METHOD) СЪЗДАТЕЛ НА ПРОДУКТИ (АБСТРАКЦИЯ)

- ```
PUBLIC ABSTRACT CLASS DISPLAYSERVICE {  
    PUBLIC VOID DISPLAY(){  
        XMLPARSER PARSER = GETPARSER();  
        STRING MSG = PARSER.PARSE();  
        SYSTEM.OUT.PRINTLN(MSG);  
    }  
    PROTECTED ABSTRACT XMLPARSER GETPARSER();
```

# МЕТОД ФАБРИКА (FACTORY METHOD) КОНКРЕТНИ СЪЗДАТЕЛ

- ```
PUBLIC CLASS ERRORXMLDISPLAYSERVICE EXTENDS DISPLAYSERVICE{  
    @OVERRIDE  
    PUBLIC XMLPARSER GETPARSER() {  
        RETURN NEW ERRORXMLPARSER();  
    }  
}
```

# МЕТОД ФАБРИКА (FACTORY METHOD) КОНКРЕТНИ СЪЗДАТЕЛ

- ```
PUBLIC CLASS FEEDBACKXMLDISPLAYSERVICE EXTENDS DISPLAYSERVICE{  
    @OVERRIDE  
    PUBLIC XMLPARSER GETPARSER() {  
        RETURN NEW FEEDBACKXML();  
    }  
}
```

# МЕТОД ФАБРИКА (FACTORY METHOD)

.....

```
DISPLAYSERVICE SERVICE = NEW FEEDBACKXMLDISPLAYSERVICE();
```

```
SERVICE.DISPLAY();
```

```
SERVICE = NEW ERRORXMLDISPLAYSERVICE();
```

```
SERVICE.DISPLAY();
```

.....

# МЕТОД ФАБРИКА (FACTORY METHOD) ИЗПОЛЗВАНЕ

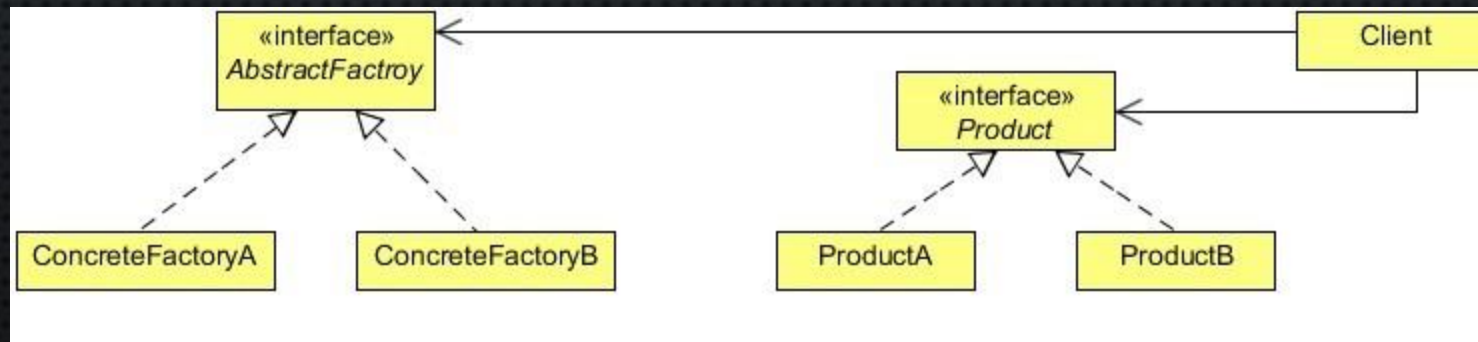
- КЛАС ИСКА НЕГОВИТЕ ПОДКЛАСОВЕ ДА ОПРЕДЕЛЯТ ОБЕКТИТЕ, КОИТО СЪЗДАВА
  - ИЗНАСЯНЕ НА ЛОГИКАТА ЗА КРИТЕРИИ
  - ЗАМЯНА НА ИЗПОЛЗВАНЕТО НА **NEW** С МЕТОД ЗА СЪЗДАВАНЕ
- ПРЕИЗПОЛЗВАНЕ НА КОД
- ИЗБЯГВАНЕ НА СИЛНА СВЪРЗАНОСТ
- SINGLE RESPONSIBILITY
- OPEN/CLOSED PRINCIPLE

# АБСТРАКТНА ФАБРИКА (ABSTRACT FACTORY)

- СЪЗДАВАНЕ НА КОЛЕКЦИИ ОТ СВЪРЗАНИ ОБЕКТИ БЕЗ УТОЧНЯВАНЕ НА ТЕХНИТЕ КОНКРЕТНИ КЛАСОВЕ

# АБСТРАКТНА ФАБРИКА (ABSTRACT FACTORY)

- АБСТРАКТНА ФАБРИКА - ДЕКЛАРИРА ИНТЕРФЕЙС ЗА ОПЕРАЦИИ, КОИТО СЪЗДАВАТ АБСТРАКТНИ ПРОДУКТИ
- ФАБРИКА ЗА КОНКРЕТНИ ПРОДУКТИ - РЕАЛИЗИРА ОПЕРАЦИИТЕ ЗА СЪЗДАВАНЕ НА КОНКРЕТНИ ПРОДУКТИ
- АБСТРАКТЕН ПРОДУКТ - ДЕКЛАРИРА ИНТЕРФЕЙС ЗА ТИП ПРОДУКТ
- КОНКРЕТЕН ПРОДУКТ - ДЕФИНИРА ПРОДУКТ, КОЙТО ДА БЪДЕ СЪЗДАДЕН ОТ СЪОТВЕТНАТА ФАБРИКА.
  - РЕАЛИЗИРА ИНТЕРФЕЙСА НА PRODUCT.
- КЛИЕНТ - ИЗПОЛЗВА САМО ИНТЕРФЕЙСИ, ДЕКЛАРИРАНИ ОТ КЛАСОВЕТЕ ABSTRACTFACTORY И PRODUCT



# АБСТРАКТНА ФАБРИКА

- ```
PUBLIC INTERFACE ABSTRACTPARSERFACTORY {  
    PUBLIC XMLPARSER GETPARSERINSTANCE(String PARSERTYPE);  
}
```

# ФАБРИКА ЗА КОНКРЕТНИ ПРОДУКТИ

- ```
PUBLIC CLASS APARSERFACTORY IMPLEMENTS ABSTRACTPARSERFACTORY {  
    @OVERRIDE  
    PUBLIC XMLPARSER GETPARSERINSTANCE(STRING PARSESTYPE) {  
        SWITCH(PARSESTYPE){  
            CASE "AERROR": RETURN NEW AERRORXMLPARSER();  
            CASE "AFEEDBACK": RETURN NEW AFEEDBACKXMLPARSER ();  
            CASE "AORDER": RETURN NEW AORDERXMLPARSER();  
            CASE "ARESPONSE": RETURN NEW ARESPONSEXMLPARSER();  
        }  
        RETURN NULL;  
    }  
}
```

# ФАБРИКА ЗА КОНКРЕТНИ ПРОДУКТИ

- ```
PUBLIC CLASS BPARSERFACTORY IMPLEMENTS ABSTRACTPARSERFACTORY {  
    @OVERRIDE  
    PUBLIC XMLPARSER GETPARSERINSTANCE(String PARSESTYPE) {  
        SWITCH(PARSESTYPE){  
            CASE "BERROR": RETURN NEW BERRORXMLPARSER();  
            CASE "BFEEDBACK": RETURN NEW BFEEDBACKXMLPARSER ();  
            CASE "BORDER": RETURN NEW BORDERXMLPARSER();  
            CASE "BRESPONSE": RETURN NEW BRESPONSEXMLPARSER();  
        }  
        RETURN NULL;  
    }  
}
```

# АБСТРАКТЕН ПРОДУКТ

- ```
PUBLIC INTERFACE XMLPARSER {  
    PUBLIC STRING PARSE();  
}
```

# КОНКРЕТЕН ПРОДУКТ

- `PUBLIC CLASS AOrderXMLParser IMPLEMENTS XMLParser{  
 @Override  
 PUBLIC STRING parse() {  
 ....`

# КОНКРЕТЕН ПРОДУКТ

- ```
PUBLIC CLASS AResponseXMLPARSER IMPLEMENTS XMLPARSER{  
    @Override  
    PUBLIC STRING PARSE() {  
        ....  
    }
```

# ДОСТАВЧИК ЗА ФАБРИКА(FACTORY-PRODUCER)

- ```
PUBLIC FINAL CLASS PARSEFACTORYPRODUCER {  
    PRIVATE PARSEFACTORYPRODUCER(){  
        THROW NEW ASSERTIONERROR();  
    }  
    PUBLIC STATIC ABSTRACTPARSEFACTORY GETFACTORY(String FACTORYTYPE){  
        SWITCH(FACTORYTYPE) {  
            CASE "AFACTORY": RETURN NEW APARSEFACTORY();  
            CASE "BFACTORY": RETURN NEW BPARSEFACTORY();  
            .....  
        }  
        RETURN NULL;  
    }  
}
```

# КАМЕНТ

- ```
ABSTRACTPARSERFACTORY PARSERFACTORY = PARSERFACTORYPRODUCER.GETFACTORY(" AFACTORY");  
XMLPARSER PARSER = PARSERFACTORY.GETPARSERINSTANCE("AORDER");  
STRING MSG="";  
MSG = PARSER.PARSE();  
  
PARSERFACTORY = PARSERFACTORYPRODUCER.GETFACTORY("BFACTORY");  
PARSER = PARSERFACTORY.GETPARSERINSTANCE("BFEEDBACK");  
MSG = PARSER.PARSE();
```

# АБСТРАКТНА ФАБРИКА ИЗПОЛЗВАНЕ

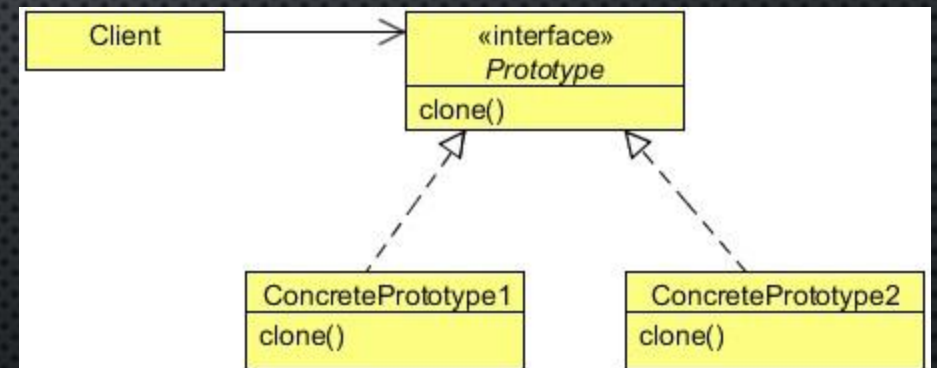
- КОГАТО СИСТЕМА ТРЯБВА ДА БЪДЕ НЕЗАВИСИМА ОТ ТОВА КАК СА СЪЗДАДЕНИ, СЪСТАВЕНИ И ПРЕДСТАВЕНИ НЕЙНИТЕ ПРОДУКТИ.
- СИСТЕМАТА ТРЯБВА ДА БЪДЕ КОНФИГУРИРАНА С ЕДНО ОТ МНОЖЕСТВОТО КОЛЕКЦИИ ПРОДУКТИ.
- КОЛЕКЦИЯТА ОТ СВЪРЗАНИ ПРОДУКТИ Е ПРОЕКТИРАНА ДА СЕ ИЗПОЛЗВАТ ЗАЕДНО И ТРЯБВА ДА СЕ НАЛОЖИ ТОВА ОГРАНИЧЕНИЕ.
- ДА СЕ ПРЕДОСТАВИ БИБЛИОТЕКА С КЛАСОВЕ ОТ ПРОДУКТИ И ТРЯБВА ДА СЕ ПРЕДОСТАВЯТ САМО ТЕХНИТЕ ИНТЕРФЕЙСИ, А НЕ ТЕХНИТЕ РЕАЛИЗАЦИИ.

# ШАБЛОН ПРОТОТИП

- СЪЗДАВАНЕ НА НОВИ ОБЕКТ НА БАЗАТА НА ПРОТОТИПЕН ЕКЗЕМПЛЯР

# ШАБЛОН ПРОТОТИП

- ПРОТОТИП — ДЕФИНИРА ИНТЕРФЕЙС ЗА КЛОНИРАНЕ
- КОНКРЕТЕН ПРОТОТИП — ИМПЛЕМЕНТАЦИЯ НА КЛОНИРАНЕТО
- КЛИЕНТ — СЪЗДАВА ОБЕКТ ЧРЕЗ ЗАЯВКА ЗА КЛОНИРАНЕ



# ШАБЛОН ПРОТОТИП ИНТЕРФЕЙС ЗА КЛОНИРАНЕ

- `PUBLIC INTERFACE PROTOTYPE EXTENDS CLONEABLE {  
 PUBLIC ACCESSCONTROL CLONE() THROWS CLONENOTSUPPORTEDEXCEPTION;  
}`

# ШАБЛОН ПРОТОТИП КОНКРЕТЕН ИНТЕРФЕЙС

- ```
PUBLIC CLASS ACCESSCONTROL IMPLEMENTS PROTOTYPE{  
    PRIVATE FINAL STRING CONTROLLEVEL;  
    PRIVATE STRING ACCESS;  
    PUBLIC ACCESSCONTROL(STRING CONTROLLEVEL,STRING ACCESS){  
        THIS.CONTROLLEVEL = CONTROLLEVEL;  
        THIS.ACCESS = ACCESS;  
    }  
  
    @OVERRIDE  
    PUBLIC ACCESSCONTROL CLONE(){  
        TRY {  
            RETURN (ACCESSCONTROL) SUPER.CLONE();  
        } CATCH (CLONENOTSUPPORTEDEXCEPTION E) {  
            E.PRINTSTACKTRACE();  
        }  
    }  
}
```

# ШАБЛОН ПРОТОТИП КОНКРЕТЕН ИНТЕРФЕЙС

```
PUBLIC STRING GETCONTROLLEVEL(){  
    RETURN CONTROLLEVEL;  
}  
PUBLIC STRING GETACCESS() {  
    RETURN ACCESS;  
}  
PUBLIC VOID SETACCESS(STRING ACCESS) {  
    THIS.ACCESS = ACCESS;  
}  
}
```

# ШАБЛОН ПРОТОТИП

- ```
PUBLIC CLASS User {  
    PRIVATE STRING USERNAME;  
    PRIVATE STRING LEVEL;  
    PRIVATE ACCESSCONTROL ACCESSCONTROL;  
    PUBLIC User(STRING USERNAME,STRING LEVEL, ACCESSCONTROL ACCESSCONTROL){  
        THIS.USERNAME = USERNAME;  
        THIS.LEVEL = LEVEL;  
        THIS.ACCESSCONTROL = ACCESSCONTROL;  
    }  
}
```

# ШАБЛОН ПРОТОТИП

```
PUBLIC ACCESSCONTROL GETACCESSCONTROL() {  
    RETURN ACCESSCONTROL;  
}
```

```
PUBLIC VOID SETACCESSCONTROL(ACCESSCONTROL ACCESSCONTROL) {  
    THIS.ACCESSCONTROL = ACCESSCONTROL;  
}
```

.....

# ШАБЛОН ПРОТОТИП

- ```
PUBLIC CLASS ACCESSCONTROLPROVIDER {  
    PRIVATE STATIC MAP<STRING, ACCESSCONTROL>MAP = NEW HASHMAP<STRING, ACCESSCONTROL>()  
    STATIC{  
        MAP.PUT("USER", NEW ACCESSCONTROL("USER","DO_WORK"));  
        MAP.PUT("ADMIN", NEW ACCESSCONTROL("ADMIN","ADD/REMOVE USERS"));  
        MAP.PUT("MANAGER", NEW ACCESSCONTROL("MANAGER","GENERATE/READ REPORTS"));  
        MAP.PUT("VP", NEW ACCESSCONTROL("VP","MODIFY REPORTS"));  
    }  
}
```

# ШАБЛОН ПРОТОТИП

```
PUBLIC STATIC ACCESSCONTROL GETACCESSCONTROLOBJECT(STRING CONTROLLEVEL){  
    ACCESSCONTROL AC = NULL;  
    AC = MAP.GET(CONTROLLEVEL);  
    IF(AC!=NULL){  
        RETURN AC.CLONE();  
    }  
    RETURN NULL;  
}  
}
```

# КЛИЕНТ

- ```
PUBLIC CLASS TESTPROTOTYPEPATTERN {  
    PUBLIC STATIC VOID MAIN(STRING[] ARGS) {  
        ACCESSCONTROL USERACCESSCONTROL = ACCESSCONTROLPROVIDER.GETACCESSCONTROLOBJECT("USER");  
        USER USER = NEW USER("User A", "USER LEVEL", USERACCESSCONTROL);  
        USERACCESSCONTROL = ACCESSCONTROLPROVIDER.GETACCESSCONTROLOBJECT("USER");  
        USER = NEW USER("User B", "USER LEVEL", USERACCESSCONTROL);  
        ....  
    }  
}
```

# ШАБЛОН ПРОТОТИП ИЗПОЛЗВАНЕ

- ИЗПОЛЗВАНЕ НА ИНСТАНЦИИ С ПРЕДВАРИТЕЛНИ ДЕФИНИРАНИ (ИЗВЕСТНИ) СЪСТОЯНИЯ
  - ЕФЕКТИВНОСТ – ИЗПОЛЗВАНЕ НА РАЗЛИЧНИ ПРОТОТИПИ ПРЕД СЪЗДАВАНЕ НА НОВА ИНСТАНЦИЯ ВСЕКИ ПЪТ
- ИЗБЯГВАНЕ НА ИЗЛИШНА ЙЕРАРХИЯ ОТ КЛАСОВЕ ЗА СЪЗДАВАНЕ С АНАЛОГИЧНА ЛОГИКА